NLP CS60075 Assignment II

How to read this document. Every project below has a Problem Statement, a list of the three datasets (full technical details for every dataset are in the supporting *Datasets* document), and three colour-coded task boxes: **blue = Task 1 (classical baseline)**, **green = Task 2 (paper reproduction)**, **purple = Task 3 (combine & improve)**. Each task box gives a one-line objective, then numbered step-by-step instructions, then the specific evaluation metrics that task must report.

What to deliver, for every task. For every one of the three tasks in the project you submit: (1) working, runnable code implementing exactly what that task's step-by-step instructions describe; (2) a results table or plot reporting every metric listed under that task's *Evaluation metrics to report*; and (3) a few sentences of discussion connecting those numbers to what the task asked you to investigate. The submitted code must be reproducible and runnable with minimal setup: **either provide a Docker-based environment with a one-command/one-click run, or provide a Google Colab notebook that can be executed end-to-end with minimal manual intervention**. Full code organization, environment, report, and submission-package requirements are given once, in full, in the *Deliverables and Submission Guidelines* section at the end of this document – read it before you start, not after you finish.

Group assignment. Each project lists three datasets. Your group will be assigned exactly *one* of the three datasets for a given project and will have to complete all three tasks (classical baseline, paper reproduction, combine & improve) using only that one dataset – you will not work on the other two.

GPU access. For projects requiring GPU resources, CS students are requested to obtain access to the CSE GPU server, with *one GPU-server account/access request per group*. The completed access-request form ias available at `https://cse.iitkgp.ac.in/archives/Server_Access_Form.pdf` and should be submitted to the Somak sir for approval. Please ensure that the form is complete and accurate before submission.

Project TA. Each group will be assigned one Project TA. The assigned TA's name, email address, and office hours will be provided to the group and should be used for project-related questions, clarification, and support.

Final deliverables. Each group is expected to submit the following:

1. **Code** – complete, working, and reproducible implementation of all three tasks, packaged using either Docker (with a one-command/one-click run) or Google Colab (with an end-to-end runnable notebook).
2. **Report** – a concise report documenting the methodology, experimental setup, results, evaluation metrics, and discussion for all three tasks.
3. **Code walkthrough demo** – a **5–10 minute** walkthrough demonstrating the implementation, explaining the main components of the code, and showing how the experiments can be run.

Final presentation before grading. Before grading, each group must give a **10-minute presentation** of its project to the assigned Project TA. The presentation should briefly cover the problem, approach, key experimental results, comparison across the three tasks, and the main conclusions. The presentation will be evaluated by the Project TA.

Assignment Submission Deadline: November 8, 2026

Project 1: N-gram Language Modeling, Smoothing, and Neural LM Reproduction

Problem Statement

A language model assigns a probability to a word sequence $w_1, \ldots, w_n$: the higher the probability, the more fluent the model considers that sequence. A classical n-gram model does this purely by counting – e.g. a trigram model estimates $P(w_i \mid w_{i-2}, w_{i-1}) \approx \text{count}(w_{i-2}, w_{i-1}, w_i) \,/\, \text{count}(w_{i-2}, w_{i-1})$ directly from a training corpus. This is simple and fully transparent, but it fails in a specific, severe way: if the exact trigram "the arctic fox" never once occurred in training, the model assigns it probability *exactly zero* – even though it is a completely ordinary English phrase. Because sentence probability is a product of such terms, one zero-probability trigram drags the *entire* sentence's probability to zero, no matter how fluent the rest of it is. This is the *data-sparsity* problem, and it worsens as n grows. *Smoothing* is the classical fix: instead of assigning zero probability to anything unseen, it reserves a small amount of probability mass for sequences the model never observed. Task 1 asks you to implement three such smoothing methods and measure directly how much each one helps.

A neural language model sidesteps this failure differently: rather than requiring an exact count-table lookup, it represents each word as a continuous vector and predicts the next word from that representation, so an unseen phrase can still receive a reasonable probability as long as its individual words and the surrounding pattern resemble things the model has seen. Task 2 reproduces a well-known such model; Task 3 then asks the project's central question – if a neural model already handles unseen sequences gracefully on its own, does classical smoothing still add anything once combined with it? Every model is evaluated with perplexity, $\text{PPL} = \exp\big(-\frac{1}{N}\sum_i \log P(w_i \mid \text{context})\big)$ (lower is better), on your assigned dataset's own test split, plus (as a bonus, common to every group regardless of assigned dataset) accuracy on BLiMP, a targeted grammaticality-judgment benchmark that tests something perplexity cannot isolate on its own: whether a model actually assigns higher probability to a grammatical sentence than to a minimally different ungrammatical one.

Datasets Used *(Full dataset details are in the Datasets document, Section 1.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **WikiText-2** — formal, encyclopedic Wikipedia text.
2. **DailyDialog** — informal, conversational text.
3. **Gutenberg (NLTK's built-in `gutenberg` corpus)** — a small collection of public-domain 19th-century literary text. Construct your own train/validation/test split (see Datasets document).

Task 1 — Baseline: Smoothed N-gram Language Model (30 marks)

Objective. Build a classical n-gram language model from raw text, implement three different smoothing methods to handle unseen word sequences, and measure how well each one predicts your assigned dataset's own held-out text – plus whether it can tell grammatical sentences from ungrammatical ones.

Step-by-step instructions:

1. **Tokenize** your assigned dataset's training split: punctuation-aware tokenizer, lowercase every token, vocabulary capped at the 30,000 most frequent tokens. Replace every token occurring fewer than 2 times in training, and every out-of-vocabulary token encountered later, with a single `<UNK>` symbol.
2. **Build count tables** for unigrams, bigrams, trigrams, and 4-grams from the tokenized training split.
3. **Implement Add-k smoothing.** Add a constant k to every count before computing probabilities. Implement this for $k \in \{1, 0.5, 0.1\}$ as three separate variants.
4. **Implement Good–Turing smoothing.** Re-estimate the probability mass for word sequences that occurred $r \leq 5$ times in training using the Good–Turing discounting formula, rather than raw counts.
5. **Implement interpolated Kneser–Ney smoothing.** Absolute discounting with discount $d = 0.75$; interpolate lower-order and higher-order n-gram estimates with a weight λ, tuned by grid search over $\{0.1, \ldots, 0.9\}$ on the *validation* split (never tune on test).
6. **Compute perplexity** of all five smoothing variants on your assigned dataset's *test* split.
7. **Compute BLiMP accuracy (bonus, same protocol for every group).** BLiMP is organized into 67 separate *paradigms*, each testing one narrow grammatical phenomenon in isolation (e.g. subject-verb agreement, or a single moved word), with 1,000 minimal sentence pairs of its own per paradigm. Select 8–10 of these 67 paradigms – spanning BLiMP's three broad categories (morphology, syntax, semantics) rather than clustered in one, since the goal is a broad grammaticality check, not deep coverage of a single phenomenon (the companion Datasets document has a ready-made starting list). Use the identical selection across Tasks 1, 2, and 3, so results stay comparable. For

your best-performing smoothing variant (by validation perplexity), compute the length-normalized log-probability your model assigns to `sentence_good` and `sentence_bad` for every pair in your selected paradigms, and score a pair correct if the model prefers `sentence_good`. Report accuracy per paradigm and overall.

Evaluation metrics to report:
- Perplexity on your assigned dataset's test split, for every smoothing variant.
- BLiMP accuracy (overall and per paradigm) for your best variant.

Task 2 — Paper Reproduction: Regularized LSTM Language Model (AWD-LSTM) (30 marks)

Objective. Reproduce a well-known regularized LSTM language model from its original paper, train it on your assigned dataset, and evaluate it with the same metrics as Task 1 so it can be compared head-to-head against your classical model.

Paper: Merity, Keskar, Socher, *Regularizing and Optimizing LSTM Language Models* ("AWD-LSTM"), arXiv:1708.02182. **Official code (reference only):** `https://github.com/salesforce/awd-lstm-lm`.

Step-by-step instructions:
1. **Build the architecture:** a 3-layer LSTM with embedding dimension 400 and hidden size 1150 (if compute-constrained, you may reduce hidden size to 400 – state this explicitly as a deviation). Tie the input embedding and output softmax matrices.
2. **Add the paper's regularization:** DropConnect (p=0.5) on hidden-to-hidden LSTM weights, embedding dropout (p=0.1), output dropout (p=0.4).
3. **Implement the optimizer:** NT-ASGD (non-monotonically-triggered averaged SGD), initial learning rate 30.
4. **Set remaining hyperparameters:** batch size 80, BPTT length 70, gradient clipping at norm 0.25.
5. **Train** on your assigned dataset's training split (the paper trains up to 500 epochs – train for approximately 50 and **explicitly state this deviation and your reasoning**).
6. **Evaluate** using the same two measurements as Task 1: perplexity on your assigned dataset's test split, and BLiMP accuracy (same protocol, same paradigms you selected in Task 1).

Evaluation metrics to report:
- Perplexity on your assigned dataset's test split.
- BLiMP accuracy (overall and per paradigm), same paradigms as Task 1.

Task 3 — Combine & Improve: Interpolated Hybrid Language Model (40 marks)

Objective. Combine your Task 1 classical model and Task 2 neural model into a single hybrid, and test directly whether the combination beats both individual models on your assigned dataset.

Step-by-step instructions:
1. **Select your best Task 1 model:** use the Kneser–Ney (KN) smoothed n-gram model as the classical half of the hybrid.
2. **Implement linear interpolation:** $P_{\text{hybrid}}(w_i \mid \text{context}) = \lambda\, P_{\text{KN}}(w_i \mid \text{context}) + (1-\lambda)\, P_{\text{AWD}}(w_i \mid \text{context})$.
3. **Tune** λ by grid search over $\{0.0, 0.1, \ldots, 1.0\}$, minimizing perplexity on your assigned dataset's *validation* split.
4. **Evaluate the hybrid** using the same two measurements as Tasks 1–2 on your assigned dataset.
5. **Build the final comparison table:** Task 1 vs. Task 2 vs. Task 3, both metrics, side by side.

Evaluation metrics to report:
- Perplexity on your assigned dataset's test split, compared against Task 1 and Task 2.
- BLiMP accuracy, compared against Task 1 and Task 2.

References

Merity, Xiong, Bradbury, Socher, *Pointer Sentinel Mixture Models*, arXiv:1609.07843 (introduces WikiText).
Merity, Keskar, Socher, *Regularizing and Optimizing LSTM Language Models*, arXiv:1708.02182 (AWD-LSTM).
Warstadt et al., *BLiMP: The Benchmark of Linguistic Minimal Pairs for English*, arXiv:1912.00582.

Project 2: Statistical vs. Neural Sequence Labeling

Problem Statement

Tagging means assigning one category label to every word in a sentence – e.g. part-of-speech (POS) tagging labels "the dog runs" as (DET, NOUN, VERB); named-entity recognition (NER) instead tags each word as part of a person/organization/location name, or not. This project builds two very different taggers, then combines them, on whichever tagging problem your assigned dataset defines.

A *Hidden Markov Model (HMM)* tagger searches for the tag sequence maximizing $P(\text{tags}) \times P(\text{words} \mid \text{tags})$, using *transition probabilities* $P(\text{tag}_i \mid \text{tag}_{i-1})$ and *emission probabilities* $P(\text{word}_i \mid \text{tag}_i)$ estimated by counting, decoded efficiently with *Viterbi decoding*. Just like the n-gram counts in Project 1, this table faces the same sparsity problem, which is why Task 1 smooths it. A neural tagger instead learns dense word representations directly from data; Task 2 reproduces a *BiLSTM* (reads the sentence both directions) that outputs an *emission score* per tag per word – the learned counterpart of the HMM's emission table – topped with a *CRF* layer that, like the HMM's transition table, scores tag-to-tag likelihood so the best whole sequence is chosen, not each word independently. Task 3 asks whether keeping the HMM's simple, inspectable Viterbi decoding but replacing its hand-counted emission table with the BiLSTM's learned scores produces a hybrid that stays transparent while closing (some of) the accuracy gap.

Datasets Used *(Full dataset details are in the Datasets document, Section 2.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **UD English EWT** — part-of-speech tagging, web-genre text (blogs, emails, reviews).
2. **UD English GUM** — part-of-speech tagging, a different genre mix (academic, fiction, spoken, how-to text).
3. **CoNLL-2003** — named-entity recognition (a different tagging problem from the two above).

Task 1 — Baseline: HMM Tagger (30 marks)

Objective. Build a first-order HMM tagger with smoothing and Viterbi decoding for whichever tagging problem your assigned dataset defines (POS for EWT/GUM, NER for CoNLL-2003).

Step-by-step instructions:

1. Build a first-order (bigram) HMM over your dataset's tagset (17 Universal POS tags for EWT/GUM, or 9 IOB2 NER tags for CoNLL-2003): MLE transition/emission probabilities with add-λ smoothing, $\lambda = 0.1$.
2. Handle unknown words with a suffix-based emission model (final 1–3 characters + capitalization), falling back to a uniform distribution over open-class tags for words your suffix model has never seen.
3. Implement Viterbi decoding in log-space.
4. Train on your assigned dataset's train split; evaluate on its test split.

Evaluation metrics to report:

- Accuracy (POS: per-token accuracy; NER: entity-level F1 via `seqeval`) on your assigned dataset's test split.

Task 2 — Paper Reproduction: Neural CRF Tagger (30 marks)

Objective. Reproduce a strong char+word BiLSTM-CRF tagger and run it on the same tagging problem as Task 1.

Paper: Lample, Ballesteros, Subramanian, Kawakami, Dyer, *Neural Architectures for Named Entity Recognition*, arXiv:1603.01360. **Code:** `https://github.com/glample/tagger`.

Step-by-step instructions:

1. Build the per-word representation: a character-level BiLSTM (hidden size 25/direction) concatenated with a 100-dim GloVe word embedding (fine-tuned during training).
2. Feed per-word representations into a word-level 1-layer BiLSTM, hidden size 100/direction, dropout = 0.5.
3. Add a linear-chain CRF output layer.
4. Train with SGD, momentum 0.9, learning rate 0.015 decayed by 0.05/epoch, gradient clip norm 5.0, up to 50 epochs, early-stopping on dev (patience 5).
5. Train on your assigned dataset's train split; evaluate on its test split.

Evaluation metrics to report:

- Accuracy (same metric convention as Task 1) on your assigned dataset's test split.

Task 3 — Combine & Improve: Neural-Emission HMM (40 marks)

Objective. Replace the Task 1 HMM's count-based emission table with the Task 2 BiLSTM's learned emission scores, keeping the HMM's transition structure and Viterbi decoder unchanged, and test whether this closes the accuracy gap.

Step-by-step instructions:

1. Take the Task 2 char+word BiLSTM encoder (without its CRF layer) and add a softmax output over tags, trained to produce calibrated per-tag emission probabilities at each position.
2. Plug these neural emission probabilities into the Task 1 HMM in place of its count-based table; keep the HMM's transition probabilities and Viterbi decoder exactly as in Task 1.
3. *(Alternative/stretch: implement a from-scratch Maximum-Entropy Markov Model using the same neural features instead.)*
4. Train/evaluate on your assigned dataset's train/test splits.
5. Build a comparison table: Task 1 vs. Task 2 vs. Task 3 on your assigned dataset.

Evaluation metrics to report:

- Accuracy on your assigned dataset's test split, compared against Task 1 and Task 2.

References

Lample et al., arXiv:1603.01360 (NAACL 2016). de Marneffe, Manning, Nivre, Zeman, *Universal Dependencies*, Computational Linguistics 47(2), 2021.

Project 3: From CKY to Neural Grammar Induction

Problem Statement

A constituency tree captures which words group together into phrases in a sentence – e.g. "the dog runs" brackets as [S [NP the dog] [VP runs]]. A Probabilistic Context-Free Grammar (PCFG) is a set of rewrite rules (e.g. S → NP VP), each with a probability; a full tree's probability is the product of every rule used to build it. Given a PCFG, the CKY algorithm finds the single highest-probability parse tree for a sentence efficiently, bottom-up. A PCFG's rule probabilities are normally estimated by counting from a treebank of gold, human-annotated trees – so the grammar is only ever as good as the (expensive, limited) supply of gold trees available. This project asks whether a grammar can instead be learned directly from raw, unlabeled sentences with no trees provided anywhere – *grammar induction* – and how the result compares to the classical, fully-supervised approach.

Task 1 builds the classical pipeline: estimate PCFG rule probabilities from your assigned partition's gold training trees, then parse with CKY. Task 2 reproduces an unsupervised approach: rather than counted rule probabilities, it introduces a continuous latent variable per sentence and predicts each rule's probability from that variable with a neural network, training the whole grammar to explain your assigned partition's training *sentences alone* – no gold tree is ever shown to it, even though (unlike a typical unsupervised-induction setup) this training corpus is necessarily small, since it is the same few thousand sentences as your partition rather than a separate, much larger raw-text corpus; this is an intentional trade-off of the one-dataset-per-group design, and you should expect Task 2's induced grammar to be weaker than published compound-PCFG results that train on far more raw text. Task 3 asks the natural next question: keep Task 2's neural rule-parameterization, but train it directly on your partition's gold trees instead of inducing it unsupervised – does a neurally-parameterized but fully-supervised grammar beat the classical, purely count-based one?

Datasets Used *(Full dataset details are in the Datasets document, Section 3.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **Penn Treebank – Partition A** — one third of a fixed, seeded 3-way random split of the 3,914-sentence NLTK Penn Treebank sample.
2. **Penn Treebank – Partition B** — a second, disjoint third of the same split.
3. **Penn Treebank – Partition C** — the remaining third of the same split.

Task 1 — Baseline: CKY with a Count-Based PCFG (30 marks)

Objective. Build a classical treebank-estimated PCFG and CKY chart parser on your assigned partition.

Step-by-step instructions:

1. Convert your assigned partition's training trees to Chomsky Normal Form: standard binarization, collapse unary chains, 0th-order markovization (repeat with 1st-order horizontal markovization as an ablation).
2. Estimate PCFG rule probabilities by relative frequency (MLE), with add-1 smoothing on any rule seen fewer than 5 times.
3. Implement the CKY chart parser ($O(n^3|G|)$).
4. Evaluate labeled and unlabeled bracketing F1 (PARSEVAL-style) on your assigned partition's test split.

Evaluation metrics to report:

- Labeled and unlabeled bracketing F1 on your assigned partition's test split.

Task 2 — Paper Reproduction: Compound PCFG (Unsupervised) (30 marks)

Objective. Reproduce an unsupervised grammar-induction model, trained only on your assigned partition's raw training sentences (gold trees withheld), and evaluate it against that same partition's gold test trees.

Paper: Kim, Dyer, Rush, *Compound Probabilistic Context-Free Grammars for Grammar Induction*, arXiv:1906.10225 (ACL 2019). **Code:** `https://github.com/harvardnlp/compound-pcfg`.

Step-by-step instructions:

1. Set nonterminals= 30, preterminals= 60, sentence-level latent $z \in \mathbb{R}^{64}$, word embedding dim= 256.
2. Build the inference-network encoder: a 1-layer BiLSTM, hidden size 256.
3. Parameterize rule probabilities via a 2-layer MLP (hidden 256, ReLU) conditioned on z.
4. Train with Adam, learning rate 1e−3, batch size 4–8 sentences, for 15–20 epochs, on your assigned partition's *training-split sentences only* – strip the gold trees before training, keeping only the raw word sequences.
5. Decode induced trees via Viterbi/minimum-Bayes-risk decoding, and evaluate against your assigned partition's gold *test* trees for bracketing F1.

Evaluation metrics to report:
- Bracketing F1 against your assigned partition's gold test trees.

Task 3 — Combine & Improve: Supervised Neural PCFG (40 marks)

Objective. Take the Task 2 neural rule-parameterization but train it with your assigned partition's gold training trees instead of inducing it unsupervised, and test whether a neurally-parameterized but fully-supervised grammar beats the classical count-based one.

Step-by-step instructions:
1. Reuse the Task 2 architecture (nonterminal/preterminal embeddings → MLP → rule probabilities) but remove the latent variable z.
2. Train this neurally-parameterized PCFG with supervised inside-outside MLE directly against your assigned partition's gold training trees.
3. Evaluate bracketing F1 on your assigned partition's test split; compare against Task 1 and Task 2.

Evaluation metrics to report:
- Bracketing F1 on your assigned partition's test split, compared against Task 1 and Task 2.

References

Kim, Dyer, Rush, arXiv:1906.10225 (ACL 2019). Marcus, Santorini, Marcinkiewicz, *Building a Large Annotated Corpus of English: The Penn Treebank*, Computational Linguistics 19(2), 1993.

Project 4: Extractive vs. Abstractive Text Summarization

Problem Statement

Summarization can be done two fundamentally different ways. *Extractive* summarization selects existing sentences from the source document – it can never misstate a fact, but is only as fluent as the sentences it picks. *Abstractive* summarization generates new sentences, potentially compressing or fusing information – more fluent, but capable of hallucinating unsupported content. Task 1 builds a classical extractive baseline, Task 2 reproduces a well-known neural abstractive model, and Task 3 asks whether extractive selection can constrain an abstractive model to stay closer to the source without losing fluency.

Datasets Used *(Full dataset details are in the Datasets document, Section 1.)*

Your group will be assigned exactly **one** of the three datasets below.

1. **CNN/DailyMail (3.0.0)** — news articles with multi-sentence, high-overlap summaries. `https://huggingface.co/datasets/abisee/cnn_dailymail` (config `"3.0.0"`), ∼287k/13k/11k train/val/test.
2. **XSum** — BBC articles with a single, highly abstractive one-sentence summary; low source overlap. `https://huggingface.co/datasets/EdinburghNLP/xsum`, ∼204k/11k/11k.
3. **SAMSum** — short messenger-style dialogues with third-person summaries; tests generalization to conversational, multi-speaker text. `https://huggingface.co/datasets/knkarthick/samsum` (`datasets.load_dataset("knkarthick/samsum")`), 14,732/818/819 train/val/test. License: CC-BY-NC-ND-4.0 (non-commercial). *(Note: the original `Samsung/samsum` repo relies on a legacy loading script that breaks under recent versions of the `datasets` library – use this parquet-based mirror instead to avoid that issue.)*

Task 1 — Baseline: TextRank / LexRank Extractive Summarizer (30 marks)

Objective. Build a graph-based extractive summarizer with no learned parameters, to see how far sentence selection alone gets on your assigned dataset.

Step-by-step instructions:

1. Sentence-split (or, for SAMSum, treat each utterance as a unit) and preprocess your dataset's splits; build per-document TF-IDF sentence vectors (IDF fit on train only) and a pairwise cosine-similarity graph.
2. Implement **TextRank**: PageRank (d=0.85) over the full graph.
3. Implement **LexRank**: same graph, but threshold weak edges (cutoff tuned on val) before ranking.
4. Select top-k sentences per document, output in original order; tune k on validation.
5. Evaluate both variants on the test split; carry the better one forward to Task 3.

Evaluation metrics to report:

- ROUGE-1/2/L F1 for both variants.
- Compression ratio and extractive density for both variants.

Task 2 — Paper Reproduction: Pointer-Generator Network with Coverage (30 marks)

Objective. Reproduce the pointer-generator summarizer, including its copy mechanism and coverage penalty, and evaluate it identically to Task 1.

Paper: See, Liu, Manning, *Get To The Point*, arXiv:1704.04368 (ACL 2017). **Code (reference):** `https://github.com/abisee/pointer-generator`. **Step-by-step instructions:**

1. Build a BiLSTM encoder (256/direction) + LSTM decoder (256) with Bahdanau attention; 128-dim embeddings from scratch; vocab capped at 50k.
2. Add the pointer mechanism: a learned p_{gen} mixing vocabulary softmax with source attention, enabling copying of OOV words.
3. Add coverage: accumulate past attention into a coverage vector, feed it into attention, and add a coverage loss (λ=1.0) penalizing repeated attention.
4. Train with Adagrad (lr 0.15), grad clip 2.0; the paper trains ∼33k iterations – if compute-constrained, train ∼3 epochs instead and **state this deviation explicitly**.
5. Decode with beam search (width 4).
6. Evaluate with Task 1's exact metrics on the test split.

Evaluation metrics to report:

- ROUGE-1/2/L F1, compared against Task 1.
- Compression ratio and extractive density, compared against Task 1.

Task 3 — Combine & Improve: Extract-then-Abstract Pipeline (40 marks)

Objective. Test whether feeding Task 2's model only Task 1's extracted sentences (instead of the full document) improves ROUGE and/or faithfulness.

Step-by-step instructions:

1. Run Task 1's better extractor to select the top-m sentences per test document (m tuned on val, larger than Task 1's k); feed only these, in order, to Task 2's trained model.
2. Evaluate ROUGE against Task 1 and Task 2 in one table.
3. Measure faithfulness: extractive density (as before), plus a factual-consistency proxy – fraction of output content words present in the *selected extract* specifically.
4. Write a short analysis: does the pipeline help more given your dataset's overlap level, and why?

Evaluation metrics to report:

- ROUGE-1/2/L F1, compared against Task 1 and Task 2.
- Extractive density and factual-consistency proxy, compared against Task 1 and Task 2.

References

Mihalcea, Tarau, *TextRank*, EMNLP 2004. Erkan, Radev, *LexRank*, JAIR 2004. See, Liu, Manning, *Get To The Point*, arXiv:1704.04368. Narayan, Cohen, Lapata, *Don't Give Me the Details, Just the Summary!*, arXiv:1808.08745 (XSum). Gliwa et al., *SAMSum Corpus*, EMNLP 2019 Workshop (SAMSum).

Project 5: From Static to Contextual – Distributional Semantics and What's Inside BERT

Problem Statement

This project studies how word representations move from classical *static* vectors to modern *contextual* representations. You will first build classical PPMI+SVD word vectors and evaluate them on semantic benchmarks (Task 1). You will then reproduce a structural probe on frozen BERT representations to determine which BERT layer most strongly encodes syntactic structure (Task 2). Finally, you will use that layer to build BERT-derived static vectors by pooling contextual representations and compare them with the classical vectors on both semantic and syntactic evaluations (Task 3).

Why this is interesting. The classical approach is based on the *distributional hypothesis*: a word's meaning can be inferred from the contexts in which it occurs. This produces one fixed vector per word. BERT instead produces a different representation for a word depending on its context. Task 2 asks whether syntactic information is recoverable from these contextual representations and which layer encodes it most strongly. Task 3 then asks whether useful information from these contextual representations survives when they are pooled into a single static vector.

What you will compare. The project evaluates two complementary properties of the representations. WordSim-353 and Google analogies measure *semantic/lexical structure*, while the dependency-tree experiments measure *syntactic structure*. The two evaluations may not agree: a representation can improve semantic performance while losing some syntactic information, or vice versa. Understanding this difference is part of the goal of the project.

Datasets Used *(Full dataset details are in the Datasets document, Section 5.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **UD English EWT** — web-genre text with gold dependency trees. Ships with its own train/validation/test split, sufficient on its own for all three tasks.
2. **UD English GUM** — a different-genre text mix with gold dependency trees. Also ships with its own train/validation/test split.
3. **UD English LinES** — a third, distinct English treebank (originally translated/parallel text) with gold dependency trees. Also ships with its own train/validation/test split.

Task 1 — Baseline: PPMI + SVD Static Vectors (30 marks)

Objective. Build the classical distributional semantics pipeline from your assigned treebank's raw text, and check, as a bonus, how much syntactic structure these vectors capture.

Step-by-step instructions:

1. Build a word-word co-occurrence matrix over your assigned dataset's training-split sentences (raw words only – ignore the dependency annotations for this step) with a symmetric window of size 5.
2. Apply PPMI weighting to the co-occurrence matrix.
3. Reduce to 100 dimensions with truncated SVD.
4. **Word-similarity evaluation:** For every one of WordSim-353's 353 word pairs, compute your model's own similarity score (cosine similarity between the two words' vectors); then measure Spearman correlation between your list of computed scores and WordSim-353's list of human-judged similarity scores, for the same 353 pairs in the same order – this tests whether your model ranks pairs as more/less similar in the same order humans did, not whether the raw numbers match. **Word-analogy evaluation:** On the Google analogy set, answer each question via the vector-offset method (excluding the three query words from the candidate answers) and report accuracy. Both evaluation sets are shared and identical across every group.
5. **Bonus structural check:** For 20 sentences sampled from your assigned dataset's test split, build a minimum spanning tree over your vectors' pairwise cosine distances between the words in each sentence, and report undirected attachment agreement against the real gold dependency tree.

Evaluation metrics to report:

- Spearman correlation on WordSim-353; accuracy on the Google analogy set.
- Undirected attachment agreement of the cosine-distance MST vs. gold trees, on your 20-sentence sample.

Task 2 — Paper Reproduction: Structural Probe on BERT (30 marks)

Objective. Reproduce a linear probe that recovers syntax trees from BERT's contextual embeddings, using your assigned dataset's gold dependency trees, and identify which layer encodes syntax most strongly.

Paper: Hewitt, Manning, *A Structural Probe for Finding Syntax in Word Representations*, NAACL 2019 (N19-1419).
Code: `https://github.com/john-hewitt/structural-probes`.

Step-by-step instructions:

1. For each layer $\ell = 1, \ldots, 12$ of `bert-base-uncased`, train a linear projection $B_\ell \in \mathbb{R}^{k \times 768}$, sweeping probe rank $k \in \{32, 64, 128\}$.
2. Train two probes per layer (distance and depth) against your assigned dataset's gold dependency distances/depths, computed from its `head` field.
3. Optimize with Adam, learning rate 1e−3, batch size 20 sentences, 30 epochs, on your assigned dataset's train split.
4. Decode an undirected tree via minimum spanning tree; score UUAS against your assigned dataset's gold *test* trees.
5. Plot UUAS and depth-probe Spearman correlation vs. layer ℓ; call the best layer ℓ^*.

Evaluation metrics to report:

- UUAS vs. layer ℓ on your assigned dataset's test split.
- Depth-probe Spearman correlation vs. layer ℓ.

Task 3 — Combine & Improve: BERT-Derived Static Vectors (40 marks)

Objective. Use the most syntax-encoding BERT layer from Task 2 to build new static vectors from your assigned dataset's text, and test whether they beat the classical ones.

Step-by-step instructions:

1. Using BERT layer ℓ^*, extract contextual embeddings for every word occurrence across your assigned dataset's training-split sentences, and average per word type.
2. Evaluate on WordSim-353/Google-analogy with the identical protocol as Task 1 (same shared benchmarks).
3. Repeat Task 1's bonus structural check on the same 20-sentence sample, now with BERT-derived vectors; compare undirected attachment agreement against Task 1.
4. Report and discuss whether pooling from the syntax-encoding layer gives better vectors than the classical pipeline on both the standard benchmarks and the structural check, and whether the two answers agree.

Evaluation metrics to report:

- Spearman correlation on WordSim-353 and Google-analogy accuracy, compared against Task 1.
- Undirected attachment agreement on the same 20 sentences, compared against Task 1.

References

Hewitt, Manning, NAACL 2019 (N19-1419). Levy, Goldberg, *Neural Word Embedding as Implicit Matrix Factorization*, NeurIPS 2014. Bommasani, Davis, Cardie, *Interpreting Pretrained Contextualized Representations via Reductions to Static Embeddings*, ACL 2020.

Project 6: A Text Processing Pipeline for Code-Mixed, Real-World Text

Problem Statement

Code-mixing is the everyday practice of freely mixing two languages within one sentence – e.g. a Hindi-English tweet like "yeh movie bahut acchi thi, must watch!" mixes Romanized Hindi words with English in the same breath. Standard NLP tools assume one language and one script throughout, so they fail immediately here. This project builds a full pipeline to handle exactly this setting, from raw mixed-script text to a sentiment label, on your assigned share of a Hindi-English code-mixed tweet corpus.

Task 1's classical pipeline has three stages: *language identification (LID)* tags each word's language; *normalization* maps the many spelling variants of Romanized Hindi ("acchi"/"achi"/"acha") to one canonical form, using a shared transliteration dictionary (Dakshina) that every group uses identically, regardless of which tweet partition they were assigned; the cleaned text is finally classified with TF-IDF features and a Random Forest. Task 2 reproduces MuRIL, a transformer pretrained specifically on Indian languages and code-mixed text, fine-tuned directly on raw, non-normalized tweets, testing how much of Task 1's normalization work a large multilingual pretrained model absorbs on its own. Task 3 tests whether the two are complementary: feed Task 1's LID predictions into Task 2's model as an extra signal.

Datasets Used *(Full dataset details are in the Datasets document, Section 6.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **SemEval SentiMix – Partition A** — one third of a fixed, seeded 3-way random split of the SemEval-2020 Task 9 Hindi–English tweet corpus. Has its own train/validation/test split, sufficient on its own for all three tasks.
2. **SemEval SentiMix – Partition B** — a second, disjoint third of the same split.
3. **SemEval SentiMix – Partition C** — the remaining third of the same split.

Task 1 — Baseline: Rule-Based LID + Random Forest (30 marks)

Objective. Build a full classical pipeline – language identification, normalization, and sentiment classification – on your assigned partition.

Step-by-step instructions:

1. Language ID: character n-gram ($n = 1$–4) TF-IDF features feeding a logistic regression (L2, $C = 1.0$), 3-way {en, hi, univ}, trained on your assigned partition's train split's gold `lang_tag` labels.
2. Normalization: build a rule-based dictionary for common Romanized-Hindi spelling variants from the shared Dakshina transliteration lexicon (identical for every group, regardless of assigned partition).
3. Downstream sentiment classifier: TF-IDF features (word + character n-grams, $n = 1$–3) over the normalized token sequence, feeding a Random Forest classifier, 300 trees, max depth 20, min samples per leaf 5.
4. Train on your assigned partition's train split; evaluate macro-F1 on its test split.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split.

Task 2 — Paper Reproduction: MuRIL Fine-Tuning (30 marks)

Objective. Reproduce a multilingual pretrained model built for Indian-language and code-mixed text, fine-tuned on your assigned partition.

Paper: Khanuja, Bansal, Mehtani, et al., *MuRIL: Multilingual Representations for Indian Languages*, arXiv:2103.10730. **Checkpoint:** `https://huggingface.co/google/muril-base-cased`.

Step-by-step instructions:

1. Fine-tune MuRIL with a 3-way sequence-classification head directly on raw (non-normalized) text from your assigned partition's train split: AdamW, learning rate 2e−5, batch size 16, 4 epochs, max sequence length 128.
2. Evaluate macro-F1 on your assigned partition's test split (same split as Task 1).
3. **Additional check:** re-run evaluation on your assigned partition's test tweets after applying Task 1's Dakshina-informed normalization at test time only (no retraining), and report whether normalizing the input helps or hurts a model fine-tuned on raw text.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split (raw input).
- Macro-F1 on the same test split with Task-1-style normalization applied at test time only.

Task 3 — Combine & Improve: LID-Augmented MuRIL (40 marks)

Objective. Feed the Task 1 language-ID tagger's predictions into the Task 2 model as an auxiliary signal, on your assigned partition.

Step-by-step instructions:

1. Concatenate a learned language-tag embedding (dim = 16) to each subword's MuRIL embedding before the transformer layers, using the Task 1 LID tagger's per-token predictions (or, more simply, prepend a language-composition summary token – state which you implemented).
2. Apply the shared Dakshina-informed normalization to the input text before tokenization.
3. Fine-tune with the same hyperparameters as Task 2, on your assigned partition's train split.
4. Evaluate macro-F1 on your assigned partition's test split; compare against Task 1 and Task 2.

Evaluation metrics to report:

- Macro-F1 on your assigned partition's test split, compared against Task 1 and Task 2.

References

Patwa et al., *SemEval-2020 Task 9*, ACL Anthology 2020.semeval-1.100. Khanuja et al., arXiv:2103.10730 (MuRIL).

Project 7: Do LLMs Know When They're Making Things Up?

Problem Statement

A language model *hallucinates* when it generates fluent, confident text that is factually wrong. Because the generated text gives no obvious surface signal that anything is wrong, detecting hallucination requires looking beyond fluency. The simplest signal is the model's own token probabilities: lower probability or higher predictive entropy when generating something it is less sure of. Task 1 builds this baseline directly, on your assigned dataset.

Task 2 reproduces a more expensive but zero-resource idea: generate several independent resamples of the same passage, and check whether the sentence being scored is *consistent* with those resamples, measured via BERTScore and an NLI contradiction probability. Task 3 combines both signals into one trained detector: split your assigned dataset's items into an 80/20 train/test partition (fixed seed), fit a logistic-regression meta-classifier on the 80% using Task 1 + Task 2's features, and evaluate on the held-out 20%. Results are reported as AUC-PR throughout (the standard metric when the interesting class – hallucinations – is a minority of examples).

Datasets Used *(Full dataset details are in the Datasets document, Section 7.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **WikiBio GPT-3 Hallucination Dataset** — GPT-3-generated biography passages with per-sentence accuracy annotations and pre-generated resamples. Sufficient on its own for all three tasks.
2. **TruthfulQA** — adversarially-constructed factual questions with correct/incorrect answer lists. Sufficient on its own for all three tasks.
3. **HaluEval (QA subset)** — question-answer pairs with a right answer and a hallucinated answer. Sufficient on its own for all three tasks.

Task 1 — Baseline: Probability-Based Hallucination Detection (30 marks)

Objective. Score every item in your assigned dataset for likely hallucination using only the model's own token probabilities.

Step-by-step instructions:

1. Using an open LLM, run a forced/greedy decoding pass over each item in your assigned dataset and compute mean token log-probability and mean predictive entropy.
2. Use these as the hallucination score (lower log-probability / higher entropy ⇒ more likely hallucinated).
3. Report sentence/answer-level AUC-PR against your assigned dataset's own accurate/inaccurate ground truth (see companion Datasets document for how each dataset defines this label).

Evaluation metrics to report:

- AUC-PR on your assigned dataset.

Task 2 — Paper Reproduction: SelfCheckGPT (30 marks)

Objective. Reproduce a zero-resource, resampling-based hallucination detector on your assigned dataset.

Paper: Manakul, Liusie, Gales, *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection*, arXiv:2303.08896.
Code: `https://github.com/potsawee/selfcheckgpt`.

Step-by-step instructions:

1. Implement SelfCheck-BERTScore: max BERTScore-F1 (backbone `roberta-large`) between each scored item and $N = 20$ resampled passages (use the dataset's own resamples if provided, as in WikiBio; otherwise generate $N = 20$ yourself at temperature 1.0, as for TruthfulQA/HaluEval).
2. Implement SelfCheck-NLI: $P(\text{contradiction})$ from `microsoft/deberta-v3-large-mnli` between each scored item and the same resamples.
3. Report AUC-PR (both variants) against your assigned dataset's ground truth.

Evaluation metrics to report:

- AUC-PR (both SelfCheck variants) on your assigned dataset.

Task 3 — Combine & Improve: Stacked Ensemble Detector (40 marks)

Objective. Combine Task 1 and Task 2's features into a single trained detector, on your assigned dataset.

Step-by-step instructions:

1. Split your assigned dataset into an 80/20 train/test partition with a fixed, stated random seed.
2. Train a logistic-regression meta-classifier on the 80% train portion, using both Task 1's features (log-probability, entropy) and Task 2's features (BERTScore-inconsistency, NLI-contradiction) as inputs.

3. Evaluate AUC-PR on the held-out 20%, comparing Task 1 alone, Task 2 alone, and this Task 3 ensemble (recomputed on the same 20% for a fair comparison).

Evaluation metrics to report:
- AUC-PR on your assigned dataset's held-out 20%, comparing Task 1, Task 2, and Task 3.

References

Manakul, Liusie, Gales, arXiv:2303.08896. Lin, Hilton, Evans, *TruthfulQA: Measuring How Models Mimic Human Falsehoods*, arXiv:2109.07958. Li et al., *HaluEval: A Large-Scale Hallucination Evaluation Benchmark*, arXiv:2305.11747.

Project 8: Chain-of-Thought Faithfulness

Problem Statement

Chain-of-thought (CoT) prompting asks a model to write out its reasoning step by step before giving a final answer, rather than jumping straight to the answer. This reliably improves accuracy – but raises a separate question: is the written reasoning actually *why* the model reached that answer, or a plausible-sounding story generated after the model had already, internally, decided some other way? Task 1 first measures the baseline this question is relative to: accuracy on your assigned reasoning dataset with no reasoning requested at all.

Task 2 reproduces the standard way of testing faithfulness on your assigned dataset, via four interventions on an already-generated chain-of-thought (early answering, adding a mistake, paraphrasing, filler tokens), each producing a faithfulness score. Task 3 uses one intervention productively: sample several independent chains-of-thought per question, use early-answering as a cheap per-sample signal for how load-bearing each one's reasoning was, and weight a majority vote toward the samples that pass this check – testing whether this improves both accuracy and aggregate faithfulness over a single, unweighted chain-of-thought, on your assigned dataset. Different groups are assigned different reasoning formats specifically so that, across the whole class, neither an accuracy nor a faithfulness claim rests on one task type.

Datasets Used *(Full dataset details are in the Datasets document, Section 8.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **GSM8K** — grade-school arithmetic word problems. Sufficient on its own for all three tasks.
2. **BIG-Bench-Hard subtask (e.g. `logical_deduction_five_objects`)** — multiple-choice logical deduction. Sufficient on its own for all three tasks.
3. **StrategyQA** — implicit multi-hop commonsense yes/no questions. Sufficient on its own for all three tasks.

Task 1 — Baseline: Direct (No-CoT) Answering (30 marks)

Objective. Establish the pre-chain-of-thought reference point on your assigned dataset: accuracy with no reasoning requested at all.

Step-by-step instructions:

1. Using an open instruction-tuned model $\geq$7B parameters, prompt for a direct final answer with no reasoning requested, in both zero-shot and 4-shot direct-answer variants.
2. Use greedy decoding (temperature = 0) throughout.
3. Sample $n = 150$–200 items from your assigned dataset with a fixed seed (see companion Datasets document for the exact count and any few-shot exemplar handling for your specific dataset).
4. Report accuracy on each dataset, for both the zero-shot and 4-shot variants.

Evaluation metrics to report:

- Accuracy on your assigned dataset (zero-shot and 4-shot direct-answer).

Task 2 — Paper Reproduction: CoT Faithfulness Interventions (30 marks)

Objective. Reproduce the standard methodology for measuring whether a model's chain-of-thought is faithful, on your assigned dataset.

Methodology (no official code exists): Lanham, Chen, Radhakrishnan, Steiner, Denison, Hernandez, et al., *Measuring Faithfulness in Chain-of-Thought Reasoning*, arXiv:2307.13702.

Step-by-step instructions:

1. Generate standard CoT responses ("let's think step by step", temperature = 0.7, top-p = 0.95, max new tokens = 512) on the same sample as Task 1; report the accuracy gain over Task 1's best direct-answer variant.
2. Apply intervention (1) *early answering*: truncate the CoT at $\{20, 40, 60, 80\}$% of its sentences and force an answer at each point.
3. Apply intervention (2) *adding a mistake*: corrupt one reasoning step and regenerate the remainder.
4. Apply intervention (3) *paraphrasing*: reword the reasoning without changing content, then re-answer.
5. Apply intervention (4) *filler tokens*: replace the reasoning with meaningless repeated tokens of the same length.
6. For every intervention, report the faithfulness score: the fraction of examples where the final answer changes as it plausibly should (or should not).

Evaluation metrics to report:

- CoT accuracy gain over Task 1 on your assigned dataset.
- Faithfulness score per intervention (4 interventions).

Task 3 — Combine & Improve: Faithfulness-Weighted Self-Consistency (40 marks)

Objective. Use Task 2's early-answering intervention as a per-sample faithfulness proxy inside a self-consistency voting scheme, on your assigned dataset.

Step-by-step instructions:

1. Sample $k = 5$ independent CoTs per question (temperature = 0.7), on the same sample as Tasks 1–2.
2. For each sampled CoT, apply Task 2's early-answering intervention at the 50% truncation point; treat a sample as more faithful/load-bearing if its answer does not change at 50% truncation.
3. Take a majority vote over the $k = 5$ final answers, weighted toward the "faithful" samples.
4. Compare accuracy of this weighted-vote pipeline against Task 1 and Task 2.
5. Apply Task 2's full set of four interventions to the final weighted-vote pipeline's output, and compare aggregate faithfulness against Task 2's single-CoT numbers.

Evaluation metrics to report:

- Accuracy on your assigned dataset, compared against Task 1 and Task 2.
- Aggregate faithfulness score (all 4 interventions), compared against Task 2.

References

Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, arXiv:2201.11903. Lanham et al., arXiv:2307.13702. Geva et al., *Did Aristotle Use a Laptop?* (StrategyQA), TACL 2021.

Project 9: Parameter-Efficient Fine-Tuning – From Hand-Engineered Features to LoRA

Problem Statement

Before large pretrained models, adapting a system to a new NLP task meant hand-designing task-specific features and training a classical classifier on top. *Fine-tuning* a pretrained transformer replaces this with one general-purpose model whose weights are directly updated – but a model like BERT has over 100 million parameters, all updated (and stored, one full copy per task) under standard fine-tuning. *Low-Rank Adaptation (LoRA)*, reproduced in Task 2, freezes the entire pretrained model and learns a much smaller update instead: for a chosen weight matrix $W \in \mathbb{R}^{d\times d}$, LoRA adds a learned correction $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times d}$ for a small rank r (e.g. $r = 8$) – so only $2dr$ new numbers are needed (often under 1% of the original weight count).

This project asks whether hand-engineered features – the pre-transformer approach from Task 1 – still add anything once a large pretrained model is cheaply adapted this way, on your assigned dataset. Task 3 combines Task 1's classical, feature-engineered model with Task 2's LoRA-tuned model and tests whether the combination beats either alone. Different groups are assigned datasets of very different length and structure (a token-level task vs. short vs. long documents) specifically so that, across the whole class, any conclusion about which approach wins can be checked against how much context is actually available, rather than treated as universal.

Datasets Used *(Full dataset details are in the Datasets document, Section 9.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **CoNLL-2003 (NER)** — a token-level sequence-labeling task. Ships with its own train/validation/test split, sufficient on its own for all three tasks.
2. **GLUE SST-2** — short, single-sentence sentiment classification. Also ships with its own train/validation split (see companion Datasets document for an important note on its `test` split).
3. **IMDB** — long, full-document movie-review sentiment classification – included because classical TF-IDF baselines are typically far more competitive on long documents than short sentences. Also ships with its own train/test split.

Task 1 — Baseline: Classical ML with Hand-Engineered Features (30 marks)

Objective. Build a strong classical baseline for your assigned dataset, using only hand-engineered features – no pretrained transformer anywhere.

Step-by-step instructions:

1. If assigned CoNLL-2003: extract hand-crafted features per token (current/previous/next word, POS tag, word shape, capitalization pattern, prefix/suffix up to length 3); train a linear-chain CRF via `sklearn-crfsuite`, L2 regularization $c_2 = 0.1$.
2. If assigned SST-2 or IMDB: extract TF-IDF features (word 1–2-grams, max features = 20,000) from the raw text; train a Random Forest classifier, 300 trees, max depth 30, min samples per leaf 5.
3. Train on your assigned dataset's train split; report entity-level F1 (CoNLL-2003) or accuracy (SST-2/IMDB) on its evaluation split.

Evaluation metrics to report:

- Entity-level F1 or accuracy (as appropriate) on your assigned dataset's evaluation split.

Task 2 — Paper Reproduction: LoRA (30 marks)

Objective. Reproduce low-rank adaptation on your assigned dataset, comparing multiple ranks against full fine-tuning.
Paper: Hu, Shen, Wallis, et al., *LoRA: Low-Rank Adaptation of Large Language Models*, arXiv:2106.09685 (ICLR 2022). **Code:** `https://github.com/microsoft/LoRA`.

Step-by-step instructions:

1. Take `bert-base-uncased` with a token-classification head (CoNLL-2003) or sequence-classification head (SST-2/IMDB).
2. Inject low-rank matrices into the query/value projections of every transformer layer; sweep rank $r \in \{4, 8, 16, 32\}$, with $\alpha = 2r$ and LoRA dropout = 0.1; freeze all base weights.
3. Train with AdamW, learning rate 3e−4, batch size 16, weight decay 0.01, for 5 epochs (NER) or 3 epochs (SST-2/IMDB).
4. Also train a full-fine-tuning reference model (same base checkpoint, learning rate 2e−5) for comparison.

5. Report entity-level F1 or accuracy for every rank r, plus the full-fine-tuning reference, and plot trainable-parameter-count vs. accuracy/F1.

Evaluation metrics to report:
- Entity-level F1 or accuracy on your assigned dataset, for every LoRA rank and the full-fine-tuning reference.

Task 3 — Combine & Improve: Classical-Feature-Augmented LoRA (40 marks)

Objective. Combine the Task 1 classical model's predictions with the Task 2 LoRA-tuned model's predictions on your assigned dataset.

Step-by-step instructions:
1. Train a meta-model that takes both Task 1's output and Task 2's output probabilities as input features (logistic-regression for SST-2/IMDB, or a linear-chain-CRF-style meta-model for CoNLL-2003).
2. *(Alternative/stretch: concatenate Task 1's hand-crafted feature vector directly into the representation LoRA fine-tunes, and fine-tune jointly.)*
3. Report entity-level F1 or accuracy for Task 1, Task 2 (best rank), and Task 3, side by side, on your assigned dataset.

Evaluation metrics to report:
- Entity-level F1 or accuracy on your assigned dataset, compared against Task 1 and Task 2 (best rank).

References

Hu et al., arXiv:2106.09685 (ICLR 2022). Lafferty, McCallum, Pereira, *Conditional Random Fields*, ICML 2001.

Project 10: Open-Domain Question Answering – From Lexical Retrieval to RAG

Problem Statement

Open-domain question answering means answering a question using a large document collection rather than a single passage handed to the model – the system must first find relevant evidence, then produce an answer from it. This splits into two sub-problems: *retrieval* and *answering*. Task 1 builds the classical pipeline for both halves on your assigned dataset: retrieval with BM25 (a lexical scoring function ranking documents by exact word overlap, weighting rare words more than common ones), and answering by extraction (selecting a span of words already present in the retrieved passage, via simple overlap heuristics).

Task 2 reproduces *Retrieval-Augmented Generation (RAG)*: dense retrieval (question and passage each encoded into a vector, ranked by similarity, so a relevant passage can match even sharing few exact words with the question), and free-form generation by an LLM conditioned on the retrieved passages. Task 3 fuses both retrieval signals into one retriever, and separately reuses Task 1's lexical-overlap idea as a *groundedness check* on Task 2's generated answers – low vocabulary overlap with any retrieved passage is a plausible warning sign the model generated the answer from its own prior knowledge rather than the retrieved evidence, connecting to the hallucination question in Project 7. Every system is scored with Exact Match, token-F1, and recall@k on your assigned dataset's own passage index – built once by your group and reused for all three tasks.

Datasets Used *(Full dataset details are in the Datasets document, Section 10.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **rag-mini-wikipedia** — a small, self-contained Wikipedia passage index with its own question-answer pairs, all guaranteed answerable from that same index. Sufficient on its own for all three tasks.
2. **Natural Questions (open)** — real search-engine questions with short gold answers, paired with a Wikipedia-paragraph index your group constructs (see companion Datasets document). Sufficient on its own for all three tasks.
3. **TriviaQA (`rc.nocontext`)** — trivia-style questions, often requiring more indirect or multi-hop reasoning than Natural Questions, paired with a Wikipedia-paragraph index your group constructs. Sufficient on its own for all three tasks.

Task 1 — Baseline: Classical Lexical Retrieve-and-Extract (30 marks)

Objective. Build a fully classical, non-neural open-domain QA pipeline on your assigned dataset.

Step-by-step instructions:

1. If assigned rag-mini-wikipedia: index the provided passage corpus directly. If assigned Natural Questions or TriviaQA: construct your own Wikipedia-paragraph index covering your sampled questions' topics (see companion Datasets document) – this index is built once by your group and reused in Tasks 2–3.
2. Retrieval: BM25 (`rank_bm25`, $k_1 = 1.5$, $b = 0.75$), top-$k = 5$.
3. Answering: no generation – select the sentence within the top-retrieved passage with the highest lexical overlap with the question, then extract the longest noun phrase or number span from it.
4. Evaluate EM/F1 and recall@k ($k \in \{1, 3, 5, 10\}$) on your assigned dataset's question set.

Evaluation metrics to report:

- EM, F1, and recall@k on your assigned dataset.

Task 2 — Paper Reproduction: Retrieval-Augmented Generation (30 marks)

Objective. Reproduce a dense-retrieval-plus-generation pipeline on the same index as Task 1.

Paper: Lewis, Perez, Piktus, et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, arXiv:2005.11401 (NeurIPS 2020). **Reference retriever:** `https://github.com/facebookresearch/DPR`.

Step-by-step instructions:

1. Dense retriever: `sentence-transformers/all-MiniLM-L6-v2`, FAISS flat index, cosine similarity, top-$k = 5$, over the same index built in Task 1.
2. Generator: an open instruction-tuned LLM, prompted with the top-k passages (truncated to 200 tokens each) plus the question, max new tokens = 64, temperature = 0.
3. Evaluate EM/F1 and recall@k on your assigned dataset's question set.

Evaluation metrics to report:

- EM, F1, and recall@k on your assigned dataset.

Task 3 — Combine & Improve: Hybrid Retrieval with Groundedness Verification (40 marks)

Objective. Fuse Task 1 and Task 2's retrieval signals, and use Task 1's lexical-overlap idea to check Task 2's generated answers for groundedness, on your assigned dataset.

Step-by-step instructions:

1. Combine Task 1's BM25 ranking and Task 2's dense ranking via Reciprocal Rank Fusion (constant = 60).
2. Compute lexical overlap between each Task-2-generated answer and its retrieved passages; flag (or abstain on) low-overlap answers as likely ungrounded.
3. Report EM/F1 and recall@k for the hybrid retriever vs. Task 1 and Task 2 alone, on your assigned dataset.
4. Report what fraction of correct generated answers have high lexical overlap with evidence, vs. what fraction of incorrect answers have low overlap.

Evaluation metrics to report:

- EM, F1, and recall@k on your assigned dataset, compared against Task 1 and Task 2.
- Groundedness-check accuracy.

References

Lewis et al., arXiv:2005.11401 (NeurIPS 2020). Chen, Fisch, Weston, Bordes, *Reading Wikipedia to Answer Open-Domain Questions* (DrQA), ACL 2017. Joshi, Choi, Weld, Zettlemoyer, *TriviaQA*, ACL 2017.

Project 11 : Metaphor Detection : Selectional-Preference Violation vs. Neural Token Classification

Problem Statement

A word is used *metaphorically* when its contextual meaning departs from its most basic, literal sense ,e.g. in "the company *digested* its rival," *digested* is not literally about a stomach. Metaphor is pervasive in ordinary language (estimates put roughly one in every seven or eight content words in running English text as metaphorical), yet it is invisible to most NLP pipelines: a parser or SRL system treats "digested" identically whether the sentence is about a stomach or a merger. Metaphor *detection* , flagging which words in a sentence are used metaphorically which is a prerequisite for any downstream system that needs to distinguish literal from figurative meaning. The classical approach formalizes a specific linguistic theory of what makes a word metaphorical: the *Selectional Preference Violation* (SPV) hypothesis holds that a word is metaphorical when it violates the typical semantic category of arguments its literal sense prefers , verbs like "digest" literally prefer a food-type object, so pairing it with an abstract object like "a rival" signals metaphor. This can be operationalized purely from corpus co-occurrence statistics, with no learned parameters. A neural approach instead treats metaphor detection as token classification: label every word in a sentence as literal or metaphorical directly from contextual embeddings, letting the model discover whatever signal separates the two classes on its own, whether or not it resembles selectional violation. Task 3 asks the project's central question – does the neural model's decision boundary actually track selectional-preference violation the way the classical theory predicts, or does it succeed (or fail) for entirely different reasons?

Datasets Used *(Full dataset details are in the Datasets document, Section 11.)*

Your group will be assigned exactly **one** of the three datasets below, and will complete all three tasks (baseline, paper reproduction, combine & improve) using *only* that dataset – you will not use the other two.

1. **VU Amsterdam Metaphor Corpus (VUA), All-POS track** , 117 documents across four registers (news, academic, fiction, conversation), every content word annotated metaphor/literal under the MIPVU protocol. The largest and most linguistically diverse of the three. Freely available from the official shared-task release: `https://github.com/EducationalTestingService/metaphor/tree/master/VUA-shared-task`.
2. **TroFi (Trope Finder) dataset** literal/nonliteral sentence pairs built around 50 ambiguous English verbs, drawn from Wall Street Journal text; a smaller, verb-only, single-register corpus. Freely available from the original SFU NatLang release, `https://natlang.cs.sfu.ca/software/trofi.html` (mirrored at `https://github.com/sfu-natlang/trofi-metaphor-data` and on Hugging Face as `CreativeLang/trofi_metaphor`).
3. **MOH-X** , a small, curated set of verb-only metaphor sentences (a subset of the Mohammad et al. MOH dataset restricted to single, unambiguous target verbs per sentence); the least noisy but also the smallest of the three , expect to discuss variance and small-sample caveats in your write-up. *Note:* MOH-X has no official standalone release page; obtain the pre-formatted version via the Google Drive links in the Gao et al. or MelBERT repositories.

Task 1 — Baseline: Selectional-Preference Violation Detector (30 marks)

Objective. Build a fully classical, non-neural metaphor detector on your assigned dataset using selectional-preference statistics only.

Step-by-step instructions:

1. Preprocess: tokenize, POS-tag, and dependency-parse your assigned dataset's train/val/test splits.
2. Build selectional-preference profiles: for every predicate governing a subject/object relation in your training split, collect the WordNet supersense of each filler and estimate $P(c \mid v)$ per predicate.
3. Compute Resnik's Selectional Preference Strength and per-token association score $\log \frac{P(c_{\text{actual}}|v)}{P(c_{\text{actual}})}$ for every content word.
4. Threshold the association score (grid search on validation, maximize F1) to output literal/metaphorical.
5. Fall back to majority-class (literal) for tokens with no governing predicate or zero co-occurrence count.
6. Evaluate on your assigned dataset's test split.

Evaluation metrics to report:

- Precision, recall, F1 (metaphor class) on your assigned dataset.
- F1 by POS (verb/noun/adj/prep).

Task 2 — Paper Reproduction: Neural Sequence Labeling for Metaphor (30 marks)

Objective. Reproduce a BiLSTM token-classification metaphor detector and evaluate it identically to Task 1.

Paper: Gao, Choi, Choi, Zettlemoyer, *Neural Metaphor Detection in Context*, arXiv:1808.09653 (EMNLP 2018).

Reference code: `https://github.com/gao-g/metaphor-in-context`.

Step-by-step instructions:

1. Input: GloVe (`glove.840B.300d`) concatenated with ELMo embeddings per token.
2. Encoder: 2-layer BiLSTM, hidden size 300/direction, dropout 0.5.
3. Output: per-token binary softmax (sequence labeling for VUA; target-word-only classification for TroFi/MOH-X).
4. Train with Adam, lr = 0.001, early stopping on validation F1 (patience 5).
5. Evaluate on your assigned dataset's test split, same metrics and POS breakdown as Task 1.

Evaluation metrics to report:

- Precision, recall, F1 (metaphor class) on your assigned dataset.
- F1 by POS, same categories as Task 1.

Task 3 — Combine & Improve: SPV-Augmented Neural Detector (40 marks)

Objective. Test whether Task 2's predictions track Task 1's selectional-violation signal, and fuse the two into a single feature-augmented model.

Step-by-step instructions:

1. Compute point-biserial correlation between Task 1's association score and Task 2's per-token correctness.
2. Retrain Task 2 with one added scalar feature per token: the Task 1 association score (plus a cold-start indicator flag).
3. Train and tune identically to Task 2 on your assigned dataset.
4. Report accuracy on the Task 1/Task 2 disagreement subset, split by which model was originally correct.

Evaluation metrics to report:

- Precision, recall, F1 on your assigned dataset, compared against Task 1 and Task 2.
- Point-biserial correlation (SPV score vs. neural correctness).
- F1 on the disagreement subset.

References

Wilks, *Making Preferences More Active*, Artificial Intelligence, 1978 (selectional preference violation as a metaphor-detection theory). Resnik, *Selectional Preference and Sense Disambiguation*, ACL SIGLEX Workshop, 1997 (SPS formula). Steen et al., *A Method for Linguistic Metaphor Identification: From MIP to MIPVU*, 2010 (VUA annotation protocol). Gao, Choi, Choi, Zettlemoyer, *Neural Metaphor Detection in Context*, EMNLP 2018, arXiv:1808.09653.

Deliverables and Submission Guidelines

Code

- Submit source code as either `.py` files or `.ipynb` Jupyter notebooks (or a mix). No other language or file format for code will be accepted.
- Organize code to mirror the Task 1 / Task 2 / Task 3 structure – three clearly separated modules or notebook sections, not one undifferentiated script, so each task's contribution is independently inspectable and runnable.
- All random seeds used for data splits, model initialization, and sampling must be fixed and stated in the code and the report.

Environment and Reproducibility

- **README.md**: environment setup; how to download/prepare all three datasets (script or exact instructions); how to run Task 1, Task 2, and Task 3 end-to-end, independently and in sequence; where results/checkpoints are written.
- **requirements.txt** (pip) *or* a conda `environment.yml` (choose one), pinning all package versions.
- Any pretrained checkpoints used (e.g. `bert-base-uncased`, `google/muril-base-cased`, GloVe vectors) must be named explicitly with source.

Report

A written report (PDF, 8–12 pages excluding references/appendix), covering, at minimum, for *every* task in the project you were assigned:

- **Method summary** for each of Task 1, Task 2, and Task 3, with any deviations from the specification explicitly justified (e.g. reduced model size due to compute constraints).
- **Quantitative analysis:** every metric specified under that task's *Evaluation metrics to report*, as tables/plots, with Task 1 and Task 2 always shown as reference points alongside Task 3's results – Task 3 is graded on the size and clarity of its improvement (or, if it does not improve, on the quality of the analysis of why not).
- **Qualitative analysis:** concrete examples (correct and incorrect) with discussion, for at least Task 2 and Task 3 – not just aggregate numbers.
- **Paper-reproduction comparison (Task 2):** a section explicitly comparing your reproduced numbers to the assigned paper's reported numbers, discussing any gap and its likely cause (data scale, compute budget, hyperparameter differences).
- **Discussion:** what worked, what didn't, and why – weighted at least as heavily as raw metric values.

Submission Package

A single archive (`.zip` or `.tar.gz`) containing: `code/` (subfolders `task1/`, `task2/`, `task3/`), `README.md`, `requirements.txt` or `environment.yml`, and `report.pdf`. Do not include large data files or model checkpoints – reference their download source in the README instead.